

Modélisation d'une assistance mécanique pour l'entraînement au badminton : détection du joueur et lancer du volant

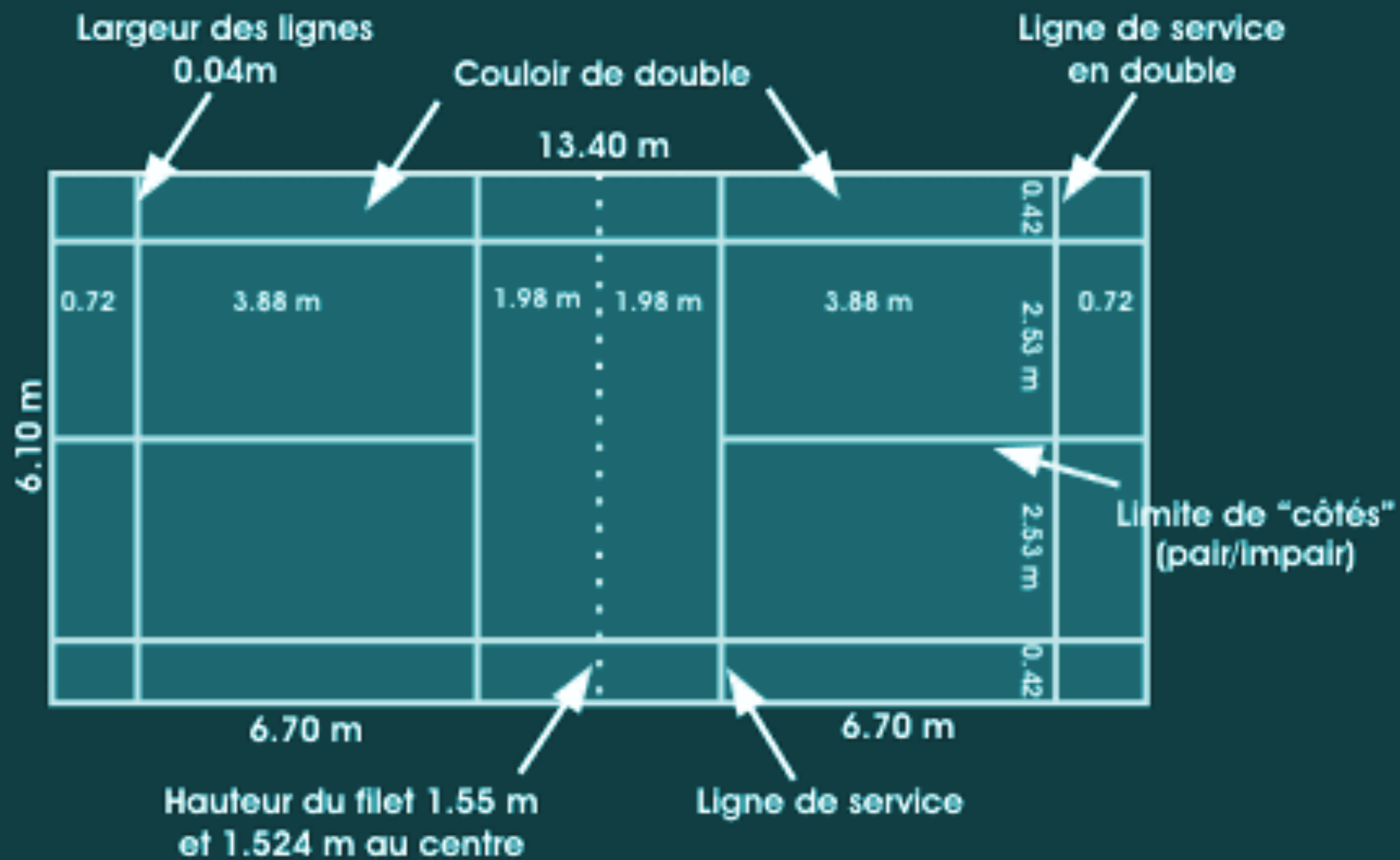
Comment modéliser un système automatisé capable de détecter la position d'un joueur de badminton et de synchroniser le tir d'un volant en transformant ces informations en commandes précises, tout en optimisant les conversions énergétiques pour garantir des tirs fiables et ajustables ?

CAPRON Aurélien
Numéro candidat : 57580

Sommaire

- I. Développement d'un algorithme de détection du joueur
- II. Conception d'un système de localisation par caméras
- III. Modélisation du lancer de volant
- IV. Analyse de la précision et des incertitudes du système

Détection du joueur





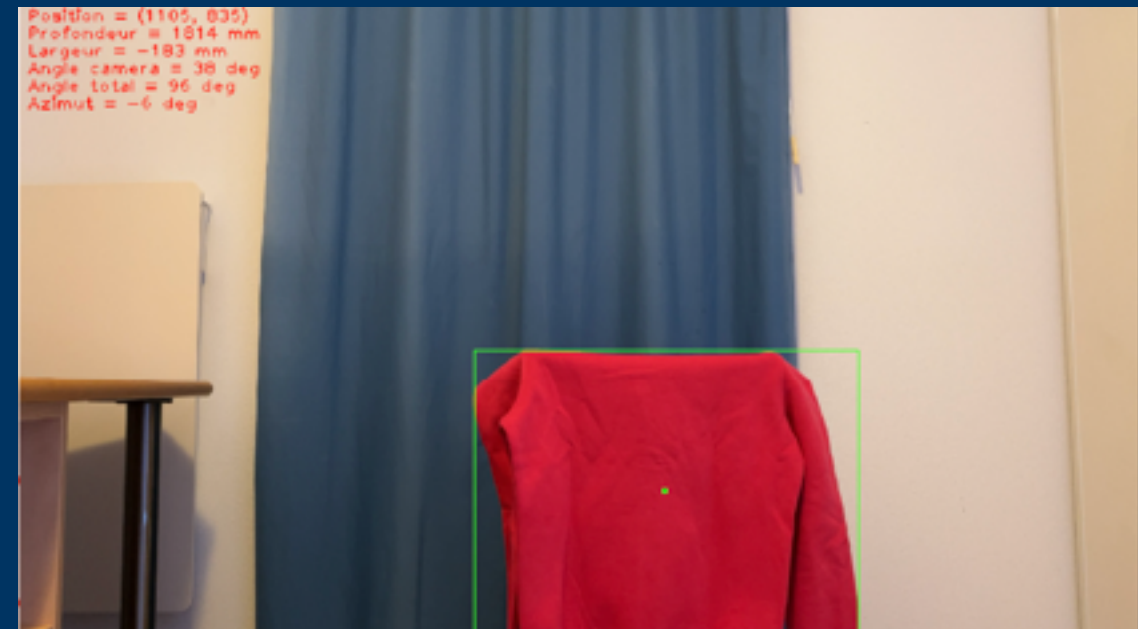
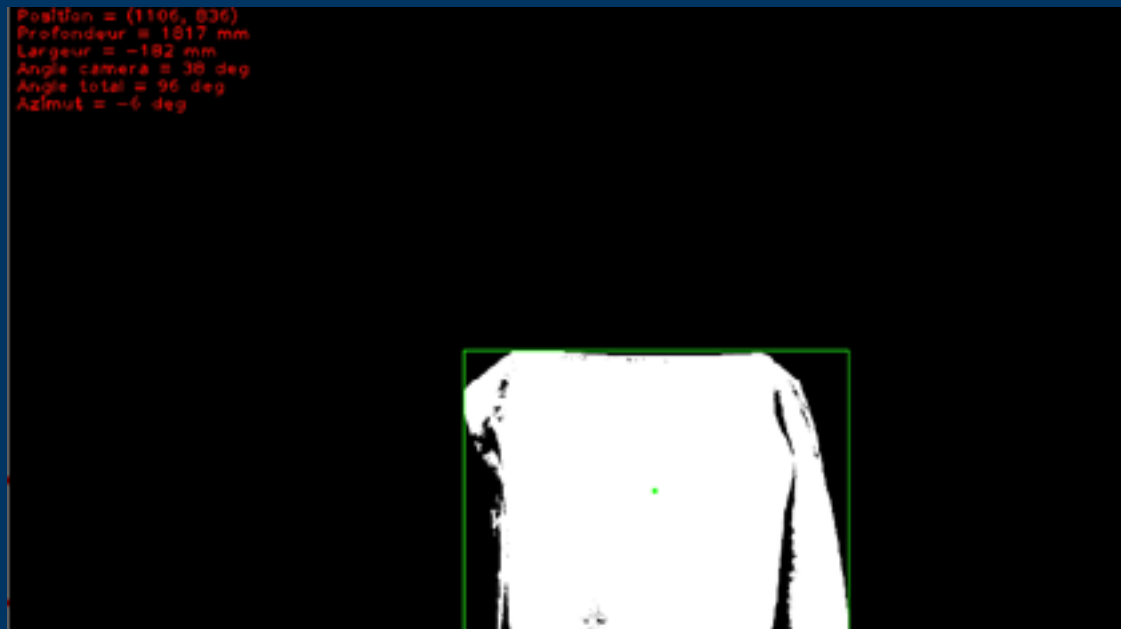
Détection par méthode YOLO

```

Detection_yolo.py > ...
1  import cv2
2  from object_detection import ObjectDetection
3
4  # Initialisation Object Detection
5  od = ObjectDetection()
6
7  def detection_objet(frame):
8      # Detection des objets sur la frame
9      (class_ids, scores, boxes) = od.detect(frame)
10     for class_id, score, box in zip(class_ids, scores, boxes):
11         class_name = classes[class_id]
12         (x, y, w, h) = box
13
14         if class_name=='person':
15             cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
16             cv2.putText(frame, class_name, (x + w, y), 0, 1, (0, 0, 255), 2)
17
18 cap = cv2.VideoCapture(0)
19 classes = od.load_class_names()
20
21 while True:
22     ret, frame = cap.read()
23     if not ret:
24         break
25
26     detection_objet(frame)
27     cv2.imshow("Telephone", frame)
28
29     key = cv2.waitKey(1)
30     if key & 0xFF == ord('q'):
31         break
32
33 cap.release()
34 cv2.destroyAllWindows()

```

Méthode de détection par filtrage de couleur



```
color = red_filter(frame, low_color, high_color) # Filtrage de la couleur rouge
threshold = threshold_filter(color) # Application du seuillage

# On ajoute un canal de couleur aux images de seuillage
threshold_bgr = cv2.cvtColor(threshold, cv2.COLOR_GRAY2BGR)

if only_player_detection:
    # Détection du joueur uniquement
    frame_figure, position = draw_figure_player(threshold, frame)
    threshold_figure = draw_figure_player(threshold, threshold_bgr)[0]
    color_figure = draw_figure_player(threshold, color)[0]
```

Hypothèses :

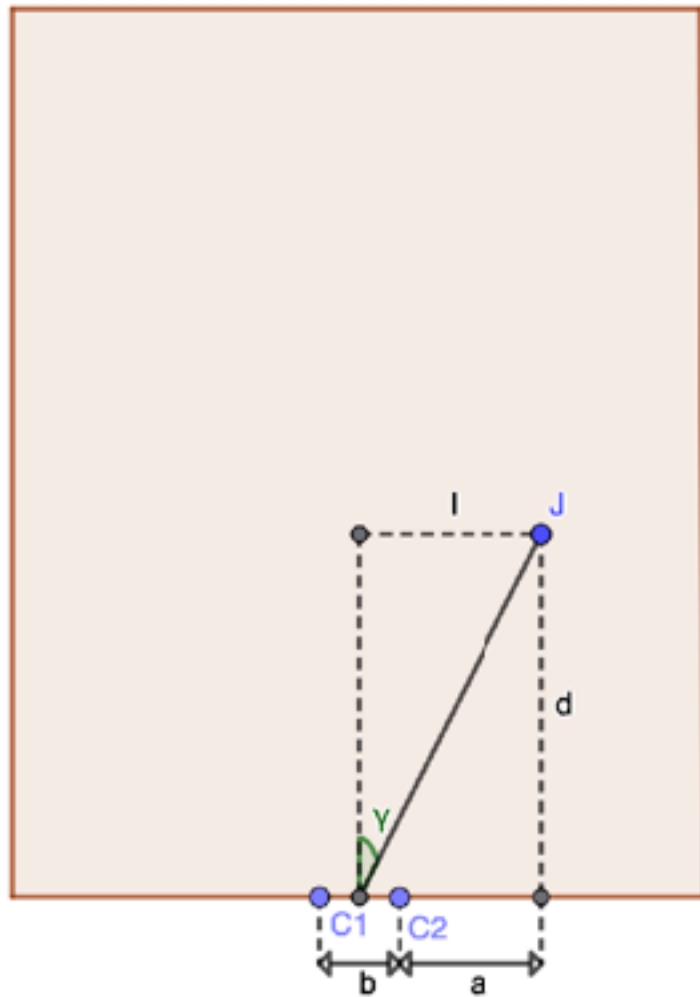
- d distance entre les deux caméras
- α et β supposés connus

$$\tan(\pi - \beta) = \frac{d}{a}$$

$$\tan(\pi - \alpha) = \frac{d}{a + b}$$

$$d = \frac{b \tan(\alpha) \tan(\beta)}{\tan(\alpha) - \tan(\beta)}$$

Détection par une
méthode de triangulation



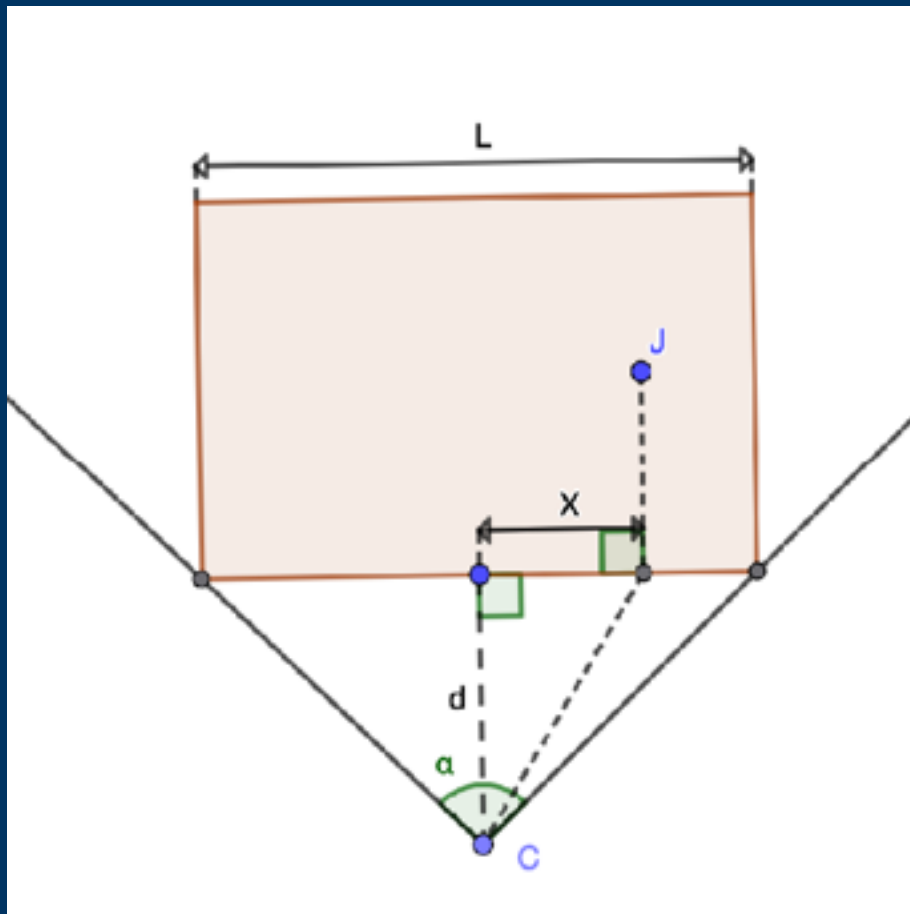
$$l = -\left(\frac{d}{\tan(\alpha)} + \frac{b}{2}\right)$$

$$\gamma = \arctan\left(\frac{l}{d}\right)$$

Détermination de l'azimut γ du joueur :

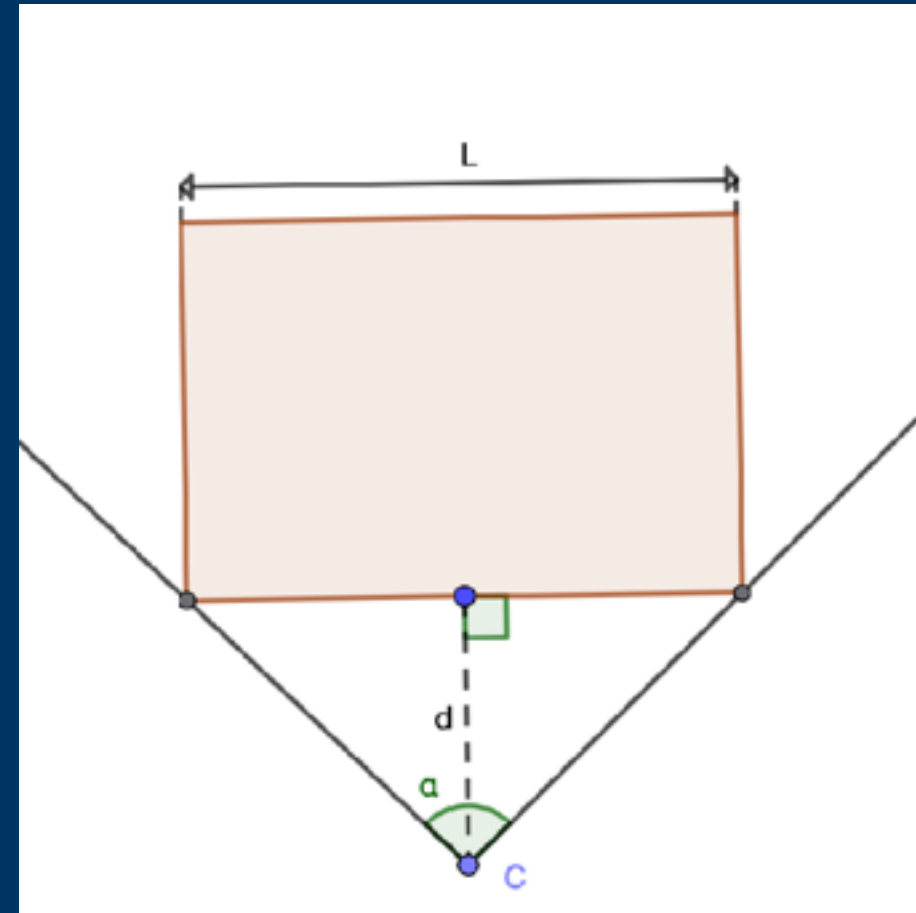
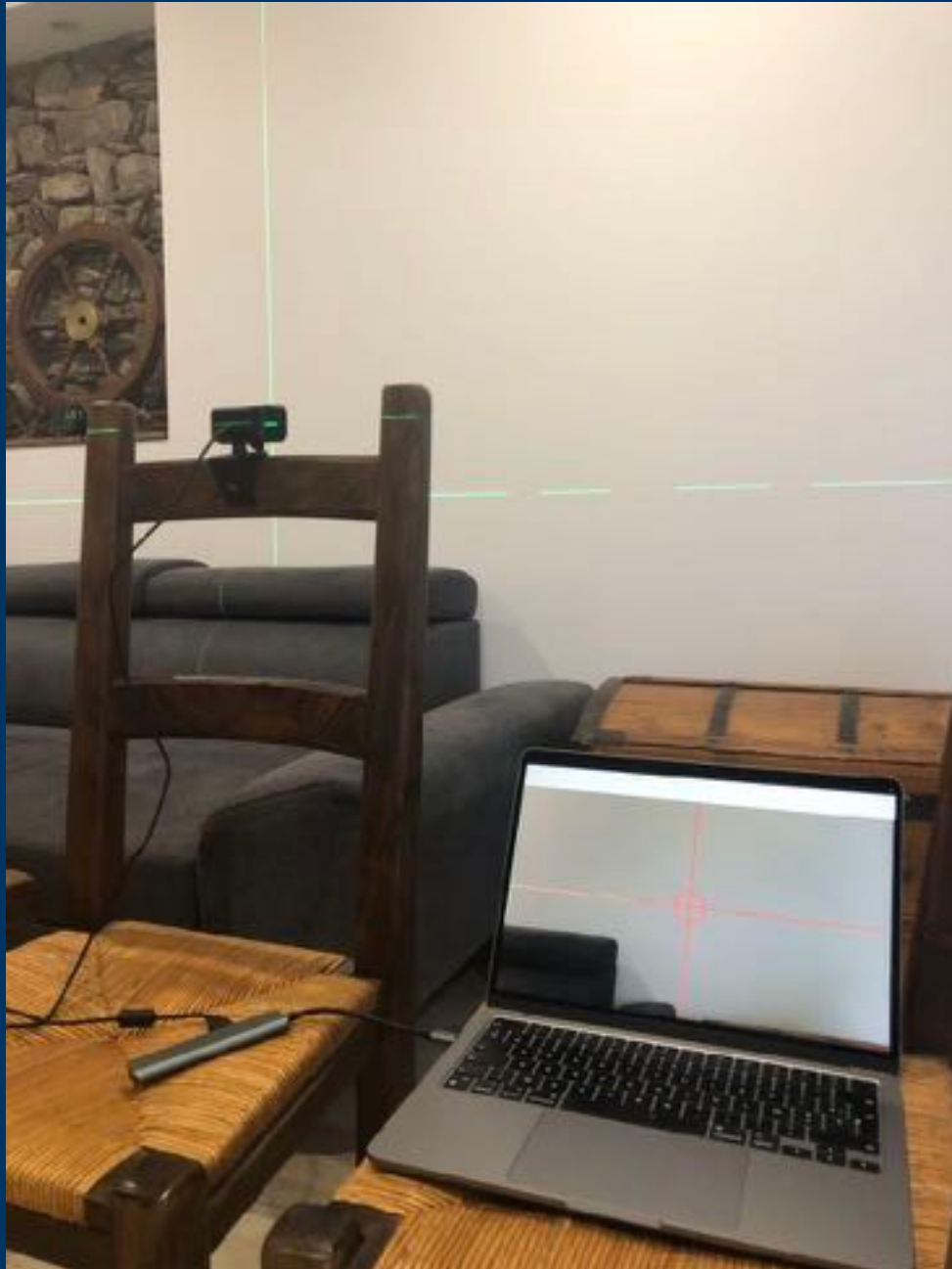
Le lancer du volant peut ainsi être assimilé à un mouvement dans un plan.

Détermination des angles α et β du joueur par rapport aux caméras



```
110 def angle_camera_player(frame, position, vision_field):
111     """
112     Calcule l'angle entre la caméra et un joueur basé sur sa position sur le cadre et l'angle de vision de la caméra.
113     """
114     focal = focal_length(frame, vision_field)
115     width_frame = frame.shape[1]
116     center_width_frame = width_frame/2
117     x = position[0]
118
119     # Calcul de l'angle basé sur la position relative du joueur par rapport au centre de l'image
120     if position[0] < center_width_frame:
121         return (vision_field/2)-np.atan((center_width_frame-x)/focal)
122     else:
123         return (vision_field/2)+np.atan((x-center_width_frame)/focal)
```

Détermination des champs de vision des caméras



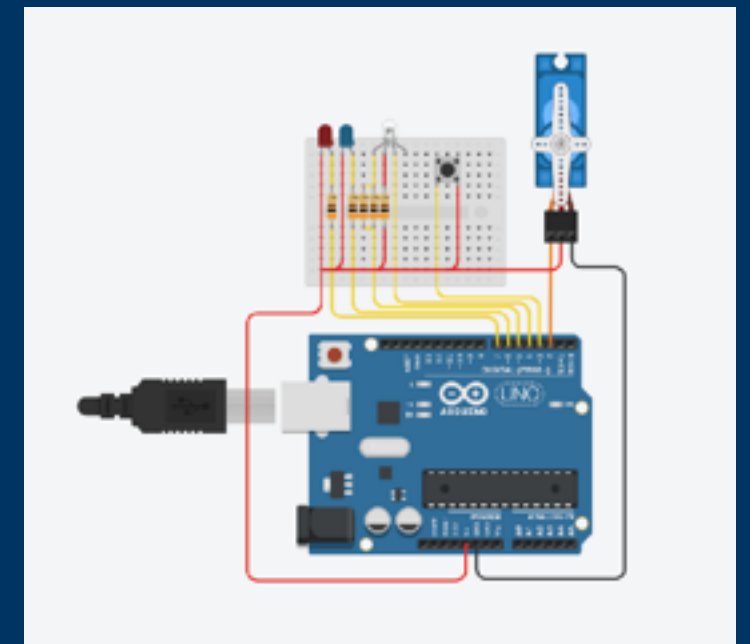
$$\alpha = 2\arctan\left(\frac{L}{d}\right)$$

Conception système caméras



Création du
support en
bois

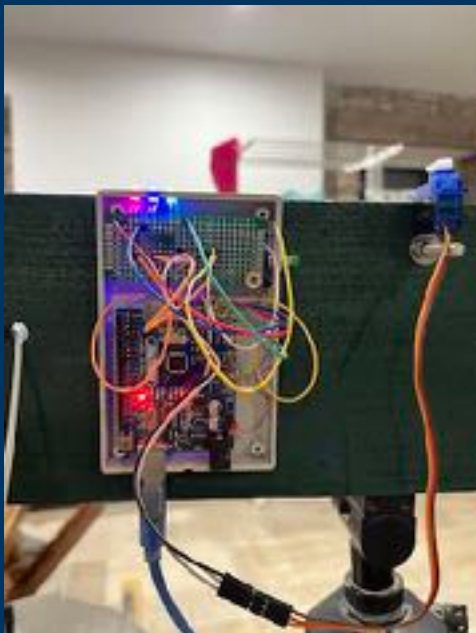
Schéma du
circuit
électronique



Programmation d'une carte Arduino UNO pour le pilotage d'un servomoteur

```
60 void loop() {
61     static String inputString = ""; // Stocke la chaîne de caractères reçue
62     static bool newData = false;    // Indique si une nouvelle donnée est
        disponible
63
64     // Lecture du message série caractère par caractère
65     while (Serial.available() > 0) {
66         char receivedChar = Serial.read();
67
68         if (receivedChar == '\n') { // Fin de message détectée
69             newData = true;
70             break;
71         } else {
72             inputString += receivedChar; // Ajout du caractère reçu à la chaîne
73         }
74     }
75 }
```

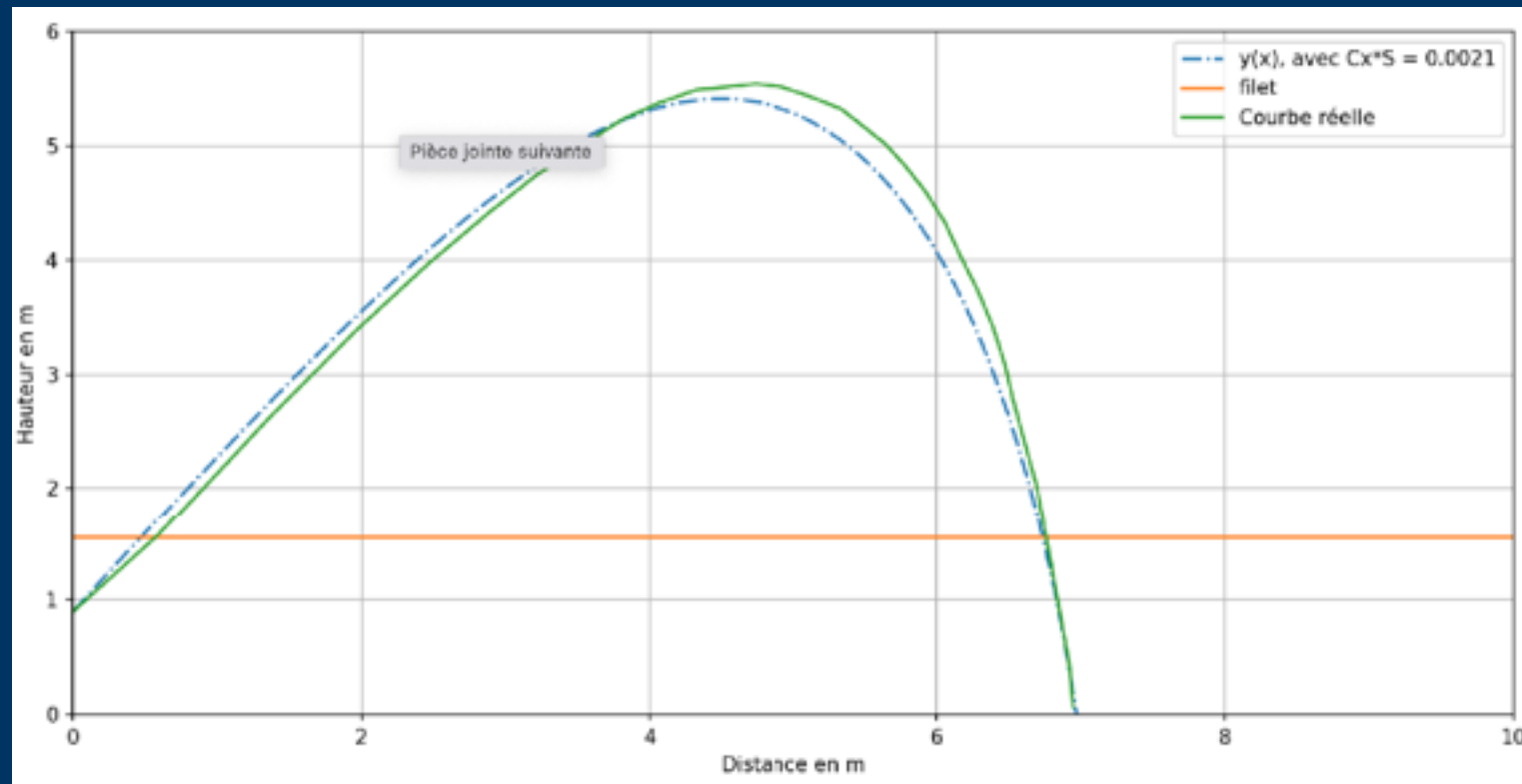
```
76 if (newData) {
77     // Conversion de la chaîne String en tableau de caractères
78     inputString.toCharArray(message, MAX_LEN_MESSAGE);
79
80     // Extraction des valeurs à partir du message reçu
81     if (sscanf(message, "%d:%d:%d:%d:%d", &azimuth, &altitude, &power, &
        frequencyThrow, &frequencyLauncher, &level) == 6) {
82
83         if (currentMillis - previousMillis > Lfrequency) {
84             previousMillis = currentMillis; // Mise à jour du dernier temps
                d'exécution
85
86             // Mise à jour de la position du servomoteur en fonction des angles
                reçus
87             SERVO1.write(45 - azimuth);
88             SERVO2.write(45 - altitude);
89         }
90
91         // Filtrage des interférences sur la fréquence
92         if ((frequencyThrow == 0) || (frequencyThrow > 99)) {
93             Lfrequency = frequencyThrow;
94         }
95
96         currentMillis = millis(); // Obtention du temps actuel
97         // Filtrage des interférences sur la fréquence
98         if ((frequencyLauncher == 0) || (frequencyLauncher > 99)) {
99             Lfrequency = frequencyLauncher;
100         }
101
102         // Mise à jour du niveau si aucune valeur précédente
103         if (level == 0) {
104             level = level;
105         }
106     } else {
107         Serial.println("Erreur de format"); // Message d'erreur si le format est
                incorrect
108     }
109
110     // Clignotement de la LED pour indiquer la réception d'un message
111     digitalWrite(LED_PIN, HIGH);
112     delay(50);
113     digitalWrite(LED_PIN, LOW);
114
115     // Réinitialisation des variables pour la prochaine lecture
116     inputString = "";
117     newData = false;
118 }
```



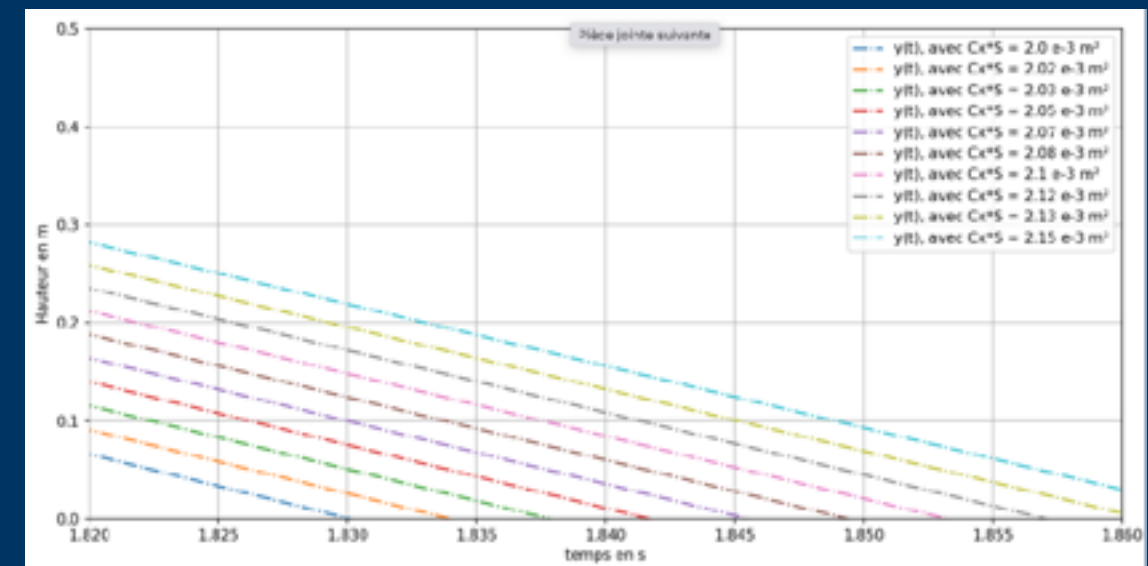
Mise en place d'une communication série entre l'Arduino et l'ordinateur via le port série.

```
# Envoi des données au microcontrôleur via le port série. Format des données envoyées : "azimut:altitude:puissance:fréquence:difficulté"
ser.write(("azimut_servo:{altitude}:{puissance}:{frequency_throw}:{frequency_launcher}:{level_difficulty}\n").encode());
```

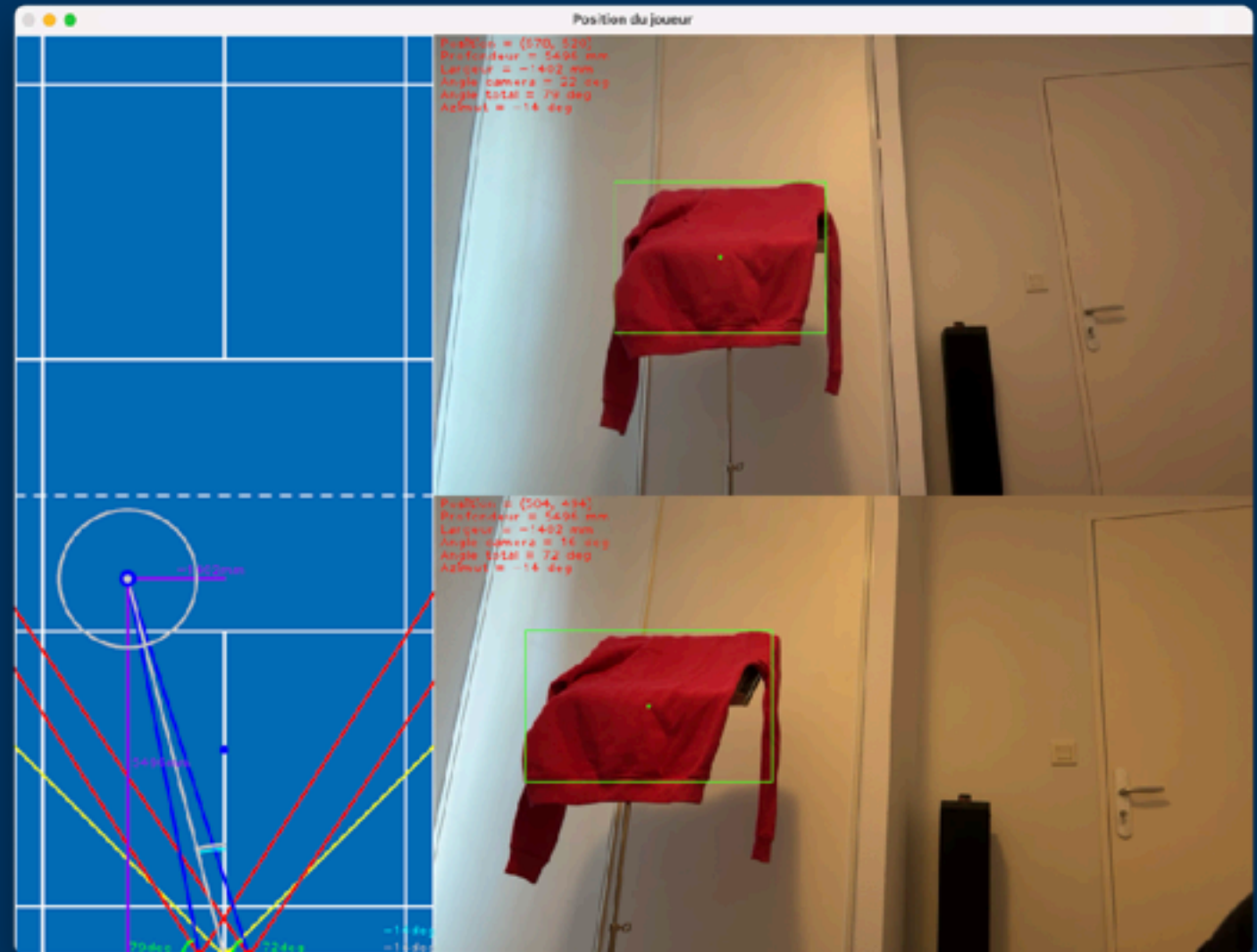
Modélisation du lancer de volant



Comparaison entre la courbe expérimentale et la courbe théorique du lancer de volant.



Analyse de la précision du système



Veuillez indiquer 1, 2 ou 3 pour changer l'affichage des caméras.

- 1 : Affichage sans modification
- 2 : Affichage du seuillage de la couleur filtrée
- 3 : Affichage seulement de la couleur filtrée

Pour modifier un paramètre dynamiquement, utilisez les préfixes suivants :

- 'V' : Ajuster la fréquence d'envoi du volant
- 'F' : Ajuster la fréquence de mis à jour du lanceur
- 'R' : Modifier le rayon de difficulté
- 'P' : Changer la profondeur de la position permanente du tir du volant
- 'L' : Changer la largeur de la position permanente du tir du volant

Libération des caméras Gauche : "iPhone" et Droite : "Webcam"

Précision des paramètres mesurés par le système

Distance du joueur mesurée (mm)	2698	5467	7412
Distance du joueur réelle (mm)	2730	5448	7388
Azimut du joueur mesuré (deg)	-10,17	7,67	23,67
Azimut du joueur réel (deg)	-10,80	7,98	24,20

Conclusion

Annexe

I. Codes Python

II. Code Arduino

Bibliothèques Python assurant le bon déroulement des programmes :

- numpy==2.2.3
- opencv-python==4.11.0.86
- pyserial==3.5

Les codes sources sont disponibles en ligne sur le site Github :
"<https://github.com/AurelienCapron/Lanceur-de-volant-de-badminton>"

I. Codes Python

Programme_principal.py

```
138 # ===== #
139 #                               6. Boucle principale                               #
140 # ===== #
141
142 # Création du terrain fictif
143 court = badminton_court(baseline_court,scope_launcher,vision_field_left,vision_field_right,color_court,color_launcher,color_camera,court_initialization)
144
145 real_condition_launcher = False # Condition réel de l'utilisation du lanceur
146
147 previous_written_value = 0 # Initialisation de la variable pour stocker la dernière valeur écrite et détecter les changements
148
149 while True:
150
151     # Capture des images des caméras
152     ret_left, frame_left = cap_left.read()
153     ret_right, frame_right = cap_right.read()
154
155     # Vérification que les deux caméras fonctionnent correctement
156     if not ret_left or not ret_right:
157         problem_camera = True
158         print("\n\033[31mProblème lors de la connexion aux caméras\033[0m\n")
159         break
160
161     # Ajustement du rayon de difficulté en fonction de l'échelle du terrain
162     radius_difficulty_court = int(radius_difficulty/scale)
163
164     # Détection du joueur sur chaque caméra
165     frame_figure_left, threshold_left, threshold_figure_left, color_left, color_figure_left, position_left = final_frame(frame_left,low_color_left,high_color_left)
166     frame_figure_right, threshold_right, threshold_figure_right, color_right, color_figure_right, position_right = final_frame(frame_right,low_color_right,high_color_right)
167
168     # Calcul de la position du joueur et des angles réels
169     depth_player,width_player,angle_left,angle_right,total_angle_left,total_angle_right,azimut = player_variable(color_figure_left,baseline,position_left,color_figure_right,baseline,position_right)
170
171     # Détermination de la position du joueur et de la difficulté sur le terrain fictif ainsi que l'azimut de la difficulté
172     position_player_court, position_player_court_base, position_difficulty_court, position_difficulty_base, azimut_difficulty = difficulty_variable(court,position_player_court_base,position_difficulty_base,azimut_difficulty)
173
174     # Détermination de la position de tir permanent sur le terrain fictif ainsi que son azimut
175     position_permanent_court, position_permanent_court_base, azimut_permanent = permanent_variable(court,position_permanent_base,azimut_permanent)
```



```

220 #----- Mise à jour de différents paramètres -----
221
222 # Récupération de la valeur modifiée de la difficulté par l'utilisateur
223 written_value = parameter.valeur_modifiee
224
225 # Analyse de l'entrée utilisateur pour déterminer s'il s'agit d'une fréquence ou d'un rayon de difficulté
226 modified_value, written_value = input_analysis(written_value)
227
228 index_modification = str(modified_value) + str(written_value)
229
230 # On vérifie que la valeur écrite est différente de la précédente
231 if index_modification != previous_written_value:
232
233     # Mise à jour des paramètres en fonction du type de valeur écrite
234
235     # Mise à jour du mode de suivi
236     if modified_value == 1:
237         tracking_mode = written_value
238         if tracking_mode in ["True", "true"]:
239             print("\033[32mActivation du mode de suivi du joueur\033[0m\n")
240         else:
241             print("\033[33mDésactivation du mode de suivi du joueur\033[0m\n")
242
243     # Mise de l'affichage des caméras
244     elif modified_value == 2:
245         if written_value == "1":
246             selected_frame = "Sans modification"
247             print("\033[36mAffichage modifié à : sans modification\033[0m\n")
248         elif written_value == "2":
249             selected_frame = "Séquence"
250             print("\033[36mAffichage modifié à : séquence\033[0m\n")
251         else:
252             selected_frame = "Couleur filtrée"
253             print("\033[36mAffichage modifié à : couleur filtrée\033[0m\n")
254
255     # Mise à jour de la fréquence d'envoi du volant
256     elif modified_value == 3:
257         frequency_throw = written_value
258         print("\033[34mModification de la fréquence d'envoi des volants à (frequency_throw)ms\033[0m\n")
259
260     # Mise à jour de la fréquence de mise à jour du lanceur
261     elif modified_value == 4:
262         frequency_launcher = written_value
263         print("\033[34mModification de la fréquence de mise à jour du lanceur à (frequency_launcher)ms\033[0m\n")
264
265     # Mise à jour du rayon de difficulté
266     elif modified_value == 5:
267         radius_difficulty = written_value
268         print("\033[34mModification du rayon de difficulté à (radius_difficulty)cm\033[0m\n")
269
270     # Mise à jour de la profondeur de la position permanente de tir
271     elif modified_value == 6:
272         position_permanent[0] = written_value
273         print("\033[34mModification de la profondeur de la position permanente de tir à (position_permanent[0])cm\033[0m\n")
274
275     # Mise à jour de la largeur de la position permanente de tir
276     elif modified_value == 7:
277         position_permanent[1] = written_value
278         print("\033[34mModification de la largeur de la position permanente de tir à (position_permanent[1])cm\033[0m\n")
279
280     previous_written_value = index_modification
281
282 if selected_frame == "Sans modification":
283     selected_frame_left, selected_frame_right = frame_figure_left, frame_figure_right
284 elif selected_frame == "Séquence":
285     selected_frame_left, selected_frame_right = threshold_figure_left, threshold_figure_right
286 elif selected_frame == "Couleur filtrée":
287     selected_frame_left, selected_frame_right = color_figure_left, color_figure_right
288
289 # Vérifie si l'utilisateur appuie sur la touche "Échap" pour quitter la boucle principale
290 if cv2.waitKey() == 27:
291     break
292

```

```

293 # ----- 7. Libération du port série et des caméras ----- #
294 # ----- 7. Libération du port série et des caméras ----- #
295
296
297 if connexion_usb:
298     ser.close()
299     print(f"Fermeture du port \033[34m(port_usb)\033[0m")
300
301 cap_left.release()
302 cap_right.release()
303 cv2.destroyAllWindows()
304
305 if not problem_camera:
306     print(f"Libération des caméras \033[34mGauche\033[0m : \033[36m(camera_left)\033[0m et \033[34mDroite\033[0m : \033[36m(camera_right)\033[0m\n")

```

Detection_joueur.py

```
1  """
2  Nom du fichier : Detection_joueur.py
3  Auteur : CAPRON Aurélien
4  Date : 14/02/2025
5  Description :
6  Ce script permet de détecter un joueur dans une image ou une vidéo en utilisant des filtres
7  de couleur et de seuillage, ainsi que des contours pour déterminer sa position sur le terrain.
8  """
9
10 import cv2
11 import numpy as np
12
13 def red_filter(frame, low_color, high_color):
14     """
15     Applique un filtre de couleur pour détecter des objets dans une certaine plage.
16
17     Paramètres :
18     frame : np.ndarray
19         Image d'entrée au format BGR.
20     low_color : tuple (int, int, int)
21         Valeurs HSV minimales pour la détection de la couleur rouge.
22     high_color : tuple (int, int, int)
23         Valeurs HSV maximales pour la détection de la couleur rouge.
24
25     Retourne :
26     np.ndarray
27         Image filtrée où seules les zones de la couleur choisie sont conservées.
28     """
29     hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV) # Conversion en HSV
30     color_mask = cv2.inRange(hsv_frame, low_color, high_color) # Création du masque pour la couleur rouge
31     color = cv2.bitwise_and(frame, frame, mask=color_mask) # Application du masque sur l'image originale
32     return color
33
34 def threshold_filter(frame):
35     """
36     Applique un seuillage binaire sur une image en niveaux de gris.
37
38     Paramètres :
39     frame : np.ndarray
40         Image d'entrée au format BGR.
41
42     Retourne :
43     np.ndarray
44         Image binaire après seuillage.
45     """
46     frame_gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # Conversion en niveaux de gris
47     threshold = cv2.threshold(frame_gris, 50, 255, cv2.THRESH_BINARY)[1] # Seuillage
48     return threshold
```

```
50 def draw_figure_color(frame_ref, frame_mod):
51     """
52     Identifie le plus grand objet détecté (probablement le joueur) et dessine un rectangle et un point central autour de chaque objet de la couleur filtrée choisie détecté.
53
54     Paramètres :
55     frame_ref : np.ndarray
56         Image de référence contenant les objets détectés (image seuillée).
57     frame_mod : np.ndarray
58         Image d'origine sur laquelle les figures seront dessinées.
59
60     Retourne :
61     tuple (np.ndarray, tuple (int, int))
62         - Image avec les rectangles et points dessinés.
63         - Coordonnées du centre de l'objet détecté.
64     """
65     contours = cv2.findContours(frame_ref, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0] # Détection des contours
66     frame_final = np.copy(frame_mod)
67
68     aire_max = 0 # Initialisation de la plus grande aire détectée
69     rect_max = (0, 0, 0, 0) # Initialisation du plus grand rectangle détecté
70
71     for cnt in contours :
72         x, y, w, h = cv2.boundingRect(cnt) # Définition du rectangle englobant
73         if w*h > aire_max : # Vérification si la zone est la plus grande trouvée
74             aire_max = w*h
75             rect_max = (x, y, w, h)
76
77         cx = int((x + x + w) / 2) # Calcul du centre en x
78         cy = int((y + y + h) / 2) # Calcul du centre en y
79
80         # Dessin du rectangle et du point central
81         cv2.rectangle(frame_final, (x, y), (x+w, y+h), (0, 255, 0), 2)
82         cv2.circle(frame_final, (cx, cy), 5, (0, 255, 0), -1)
83
84     x_max, y_max, x_min, y_min = rect_max # Récupération des coordonnées du plus grand rectangle
85
86     cx_max = int((x_max + x_min + w_max) / 2) # Calcul du centre en x
87     cy_max = int((y_max + y_min + h_max) / 2) # Calcul du centre en y
88
89     return frame_final, (cx_max, cy_max)
```

```

91 def draw_figure_player(frame_ref, frame_mod):
92     """
93     Identifie le plus grand objet détecté (probablement le joueur) et dessine un rectangle autour.
94
95     Paramètres :
96     frame_ref : np.ndarray
97         Image de référence contenant les objets détectés (image seuillée).
98     frame_mod : np.ndarray
99         Image d'origine sur laquelle les figures seront dessinées.
100
101     Retourne :
102     tuple (np.ndarray, tuple (int, int))
103         - Image avec le rectangle et le point central du joueur.
104         - Coordonnées du centre du joueur détecté.
105     """
106     contours = cv2.findContours(frame_ref, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0] # Détection des contours
107     frame_final = np.copy(frame_mod)
108
109     aire_max = 0 # Initialisation de la plus grande aire détectée
110     rect_max = (0,0,0,0) # Initialisation du plus grand rectangle détecté
111
112     for cnt in contours :
113         x,y,w,h = cv2.boundingRect(cnt) # Définition du rectangle englobant
114         if w*h > aire_max : # Vérification si la zone est la plus grande trouvée
115             aire_max = w*h
116             rect_max=(x,y,w,h)
117
118     x,y,w,h = rect_max # Récupération des coordonnées du plus grand rectangle
119
120     cx = int((x + x + w) / 2) # Calcul du centre en x
121     cy = int((y + y + h) / 2) # Calcul du centre en y
122
123     # Dessin du rectangle et du point central
124     cv2.rectangle(frame_final, (x,y), (x+w,y+h), (0, 255, 0), 2)
125     cv2.circle(frame_final, (cx,cy), 5, (0, 255, 0), -1)
126
127     return frame_final, (cx,cy)

```

```

129 def final_frame(frame, low_color, high_color, only_player_detection):
130     """
131     Traite une image pour détecter le joueur et les objets rouges.
132
133     Paramètres :
134     frame : np.ndarray
135         Image d'entrée au format BGR.
136     low_red : tuple (int, int, int)
137         Valeurs HSV minimales pour la détection de la couleur rouge.
138     high_red : tuple (int, int, int)
139         Valeurs HSV maximales pour la détection de la couleur rouge.
140     only_player_detection : bool
141         Indique si on ne doit détecter que le joueur (True) ou tous les objets rouges (False).
142
143     Retourne :
144     tuple (np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, tuple (int, int))
145         - Image finale avec le joueur ou les objets détectés.
146         - Image seuillée en niveaux de gris.
147         - Image seuillée avec les figures dessinées.
148         - Image filtrée pour la couleur rouge.
149         - Image filtrée avec les figures dessinées.
150         - Position du joueur ou de l'objet détecté.
151     """
152     color = red_filter(frame, low_color, high_color) # Filtrage de la couleur rouge
153     threshold = threshold_filter(color) # Application du seuillage
154
155     # On ajoute un canal de couleur aux images de seuillage
156     threshold_bgr = cv2.cvtColor(threshold, cv2.COLOR_GRAY2BGR)
157
158     if only_player_detection:
159         # Détection du joueur uniquement
160         frame_figure, position = draw_figure_player(threshold, frame)
161         threshold_figure = draw_figure_player(threshold, threshold_bgr)[0]
162         color_figure = draw_figure_player(threshold, color)[0]
163     else:
164         # Détection de tous les objets rouges
165         frame_figure, position = draw_figure_color(threshold, frame)
166         threshold_figure = draw_figure_color(threshold, threshold_bgr)[0]
167         color_figure = draw_figure_color(threshold, color)[0]
168
169     return frame_figure, threshold, threshold_figure, color, color_figure, position

```

Variables_positions.py

```
1  """
2  Nom du fichier : Variables_positions.py
3  Auteur : CAPRON Aurélien
4  Date : 14/02/2023
5  Description :
6  Ce script contient des fonctions permettant de calculer diverses variables relatives à un joueur sur un terrain de badminton,
7  comme la profondeur, la largeur, l'angle de la caméra, et la position du joueur. Il inclut aussi des calculs pour la difficulté du jeu
8  et la projection de la position du joueur sur le terrain en fonction de la vision de la caméra et d'autres paramètres.
9  """
10
11 import numpy as np
12 from random import uniform
13 from math import sqrt
14
15 dimension_real = [13800,6300,3] # Dimensions réelles du terrain (en m)
16 dimension_representation = [2560,904,3] # Dimensions de l'image de représentation (en pixels), la hauteur est déterminée par la hauteur de deux images de taille (1800, 1024, 3)
17
18 #----- Position du lanceur et des caméras -----#
19
20 position_launcher = [5000,5000] # Position du lanceur par rapport à la base du terrain (en m) : (abscisse, ordonnée)
21 position_cameras = [3000,3000] # Position de l'installation caméras par rapport à la base du terrain (en m) : (abscisse, ordonnée)
22
23 abscissa_difference = position_cameras[0] - position_launcher[0]
24 ordinate_difference = position_cameras[1] - position_launcher[1]
25
26 def dimension_scale():
27     """
28     Calcule le facteur d'échelle entre les dimensions réelles et les dimensions représentées du terrain.
29
30     L'échelle est obtenue en divisant la première dimension réelle du terrain
31     par la première dimension de sa représentation fictive.
32     """
33     return dimension_real[0]/dimension_representation[0]
34
35 def random_point_circle(depth, width, level, radius):
36     """
37     But : Générer un point aléatoire à l'intérieur d'un cercle pour simuler la dispersion des tirs.
38     """
39     # Génération d'un rayon aléatoire avec une distribution uniforme
40     if level == 2:
41         radius = radius * sqrt(uniform(0, 1))
42
43     # Génération d'un angle aléatoire entre 0 et 2π
44     random_angle = uniform(0, 2 * np.pi)
45
46     # Conversion en coordonnées cartésiennes
47     x = width + radius * np.cos(random_angle)
48     y = depth + radius * np.sin(random_angle)
49
50     return int(x), int(y)
```

```
52 def circle_difficulty_on_court(depth_player,width_player,position_launcher,radius_difficulty):
53     """
54     But : Renvoie True si une partie du cercle de difficulté est dans le terrain, sinon le point le plus proche dans le terrain.
55     """
56     depth = False
57     width = 0
58
59     depth_launcher_court = dimension_real[0]-position_launcher[1]
60     width_left_launcher_court = position_launcher[0]
61     width_right_launcher_court = dimension_real[1]-position_launcher[0]
62
63     if depth_player-radius_difficulty > depth_launcher_court:
64         depth = True
65     if width_player >= 0 and width_player-radius_difficulty > width_right_launcher_court:
66         width = 1
67     elif width_player < 0 and abs(width_player)-radius_difficulty > width_left_launcher_court:
68         width = -1
69
70     if width == -1 and depth:
71         return (-width_left_launcher_court,depth_launcher_court)
72     elif width == 1 and depth:
73         return (width_right_launcher_court,depth_launcher_court)
74     elif width == -1:
75         return (-width_left_launcher_court,depth_player)
76     elif width == 1:
77         return (width_right_launcher_court,depth_player)
78     elif depth == True:
79         return (width_player,depth_launcher_court)
80     return True
```

```

82 # ===== #
83 # 1. Caractéristiques des caméras #
84 # ===== #
85
86 def field_of_view(distance_camera_screen,screen_length):
87     """
88     Calcule l'angle de champ de vision (FOV) moyen d'une caméra en fonction des distances de caméra et des longueurs d'écran.
89     """
90     n = len(distance_camera_screen)
91     FOV = []
92     for i in range(n):
93         # Calcul de l'angle FOV pour chaque distance et longueur d'écran
94         FOV.append(2*np.atan(screen_length[i]/(2*distance_camera_screen[i])))
95     return np.average(FOV) # Retourne la moyenne des angles FOV
96
97 def focal_length(frame,vision_field):
98     """
99     Calcule la distance focale d'une caméra à partir de la largeur de l'écran et de son champ de vision.
100     """
101     focal_length = frame.shape[1]/(2*np.tan(vision_field/2))
102     return focal_length

```

```

104 # ===== #
105 # 2. Calculs des paramètres du joueur #
106 # ===== #
107
108 #----- Calculs des angles -----
109
110 def angle_camera_player(frame,position,vision_field):
111     """
112     Calcule l'angle entre la caméra et un joueur basé sur sa position sur le cadre et l'angle de vision de la caméra.
113     """
114     focal = focal_length(frame,vision_field)
115     width_frame = frame.shape[1]
116     center_width_frame = width_frame/2
117     x = position[0]
118
119     # Calcul de l'angle basé sur la position relative du joueur par rapport au centre de l'image
120     if position[0] < center_width_frame:
121         return (vision_field/2)+np.atan((center_width_frame-x)/focal)
122     else:
123         return (vision_field/2)+np.atan((x-center_width_frame)/focal)
124
125 def angle_base_camera(vision_field):
126     """
127     Calcule l'angle de la caméra par rapport à l'axe de référence basé sur le champ de vision.
128     """
129     return (np.pi-vision_field)/2
130
131 def total_angle(angle_camera_player,angle_base_camera):
132     """
133     Calcule l'angle total entre la caméra et un joueur, en prenant en compte l'angle de la caméra et l'angle de base.
134     """
135     return angle_camera_player + angle_base_camera
136
137 def angle_position_launcher(depth_player,width_player):
138     """
139     Calcule l'azimut du joueur par rapport à la caméra (l'angle horizontal entre le joueur et la ligne de visée de la caméra).
140     """
141     if depth_player == 0 :
142         return 0
143     return np.atan(width_player/depth_player)

```

```

145 #----- Calculs de la profondeur/largeur en fonction de l'installation des caméras -----
146
147 def depth_cameras(angle_left,angle_right,baseline):
148     """
149     Calcule la profondeur par rapport aux caméras d'un joueur sur le terrain en utilisant les angles de caméra gauche et droite, ainsi que la distance de base entre les caméras.
150     """
151     tal = np.tan(angle_left)
152     tar = np.tan(angle_right)
153
154     return (baseline*tal*tar)/(tal-tar)
155
156 def width_cameras(angle_left,angle_right,baseline):
157     """
158     Calcule la largeur par rapport aux caméras de joueur (distance horizontale) à partir des angles de la caméra gauche et droite.
159     """
160     tal = np.tan(angle_left)
161     bl = baseline/2
162     depth = depth_cameras(angle_left,angle_right,baseline)
163
164     return -(depth/tal)+bl
165
166 #----- Calculs de la profondeur en fonction du lanceur -----
167
168 def depth_launcher(angle_left,angle_right,baseline,ordinate_difference):
169     """
170     Calcule la profondeur par rapport au lanceur d'un joueur sur le terrain en utilisant les angles de caméra gauche et droite, ainsi que la distance de base entre les caméras.
171     """
172     depth_cam = depth_cameras(angle_left,angle_right,baseline)
173
174     return depth_cam + ordinate_difference
175
176 def width_launcher(angle_left,angle_right,baseline,abscissa_difference):
177     """
178     Calcule la largeur par rapport au lanceur du joueur (distance horizontale) à partir des angles de la caméra gauche et droite.
179     """
180     width_l = width_cameras(angle_left,angle_right,baseline)
181
182     return width_l + abscissa_difference

```

Determination_filtre.py

```
169 #----- Fonctions pour obtenir un filtre de couleur du pixel cliqué -----
170
171 # Valeur HSV initiale, utilisée comme point de départ pour le filtrage des couleurs
172 hsv_pixel = np.array([170, 215, 105])
173
174 def get_hsv_color(event, x, y, flags, param):
175     """
176     Fonction de rappel (callback) pour récupérer la couleur HSV d'un pixel cliqué.
177
178     Lorsqu'un clic gauche est détecté sur l'image affichée, cette fonction :
179     - Récupère la couleur du pixel correspondant en format BGR.
180     - Convertit cette couleur en format HSV.
181     - Stocke la valeur HSV dans une variable globale.
182     """
183     global hsv_pixel # Utilisation d'une variable globale pour stocker la valeur HSV du pixel cliqué
184
185     if event == cv2.EVENT_LBUTTONDOWN: # Vérifie si un clic gauche est effectué
186         frame = param[0] # Récupère la dernière image capturée
187         pixel = frame[y, x] # Récupère la couleur BGR du pixel cliqué aux coordonnées (x, y)
188
189         # Convertit la couleur du format BGR en HSV et stocke le résultat dans hsv_pixel
190         hsv_pixel = cv2.cvtColor(np.uint8([pixel]), cv2.COLOR_BGR2HSV)[0][0]
```

```
192 def filter_determination(capture, camera_name):
193     """
194     Fonction permettant de déterminer un filtre HSV basé sur un clic de souris.
195
196     Cette fonction :
197     - Permet de cliquer sur un pixel pour récupérer sa couleur HSV.
198     - Applique un filtre autour de cette couleur avec une plage définie ('range_value').
199     - Affiche l'image originale et l'image filtrée en temps réel.
200     - Quitte lorsque la touche 'Échap' (ESC) est pressée.
201     """
202     global hsv_pixel # Variable globale contenant la valeur HSV sélectionnée
203
204     cv2.namedWindow(camera_name) # Crée une fenêtre d'affichage OpenCV
205
206     cv2.createTrackbar("Largeur filtre i", camera_name, 0, 100, nothing) # Crée une trackbar pour ajuster la plage HSV
207     cv2.setTrackbarPos("Largeur filtre i", camera_name, 50) # Modifie la position initiale à 50
208
209     while True:
210         ret, frame = capture.read() # Capture une nouvelle image depuis la caméra
211         if not ret:
212             print(f"\n!!! Erreur: impossible de lire la frame de la caméra cv2.VideoCapture(camera_name) !!!")
213             break
214
215         # Associer la fonction de récupération de couleur HSV au clic de souris
216         cv2.setMouseCallback(camera_name, get_hsv_color, [frame])
217
218         # Récupérer la valeur du trackbar (plage HSV ajustée par l'utilisateur)
219         range_value = cv2.getTrackbarPos("Largeur filtre i", camera_name)
220
221         # Récupération des composants H, S et V de la couleur sélectionnée et conversion en entier
222         h, s, v = int(hsv_pixel[0]), int(hsv_pixel[1]), int(hsv_pixel[2])
223
224         # Détermination des bornes inférieures, en s'assurant qu'elles restent dans les limites (min 0)
225         h_min = max(h - range_value, 0)
226         s_min = max(s - range_value, 0)
227         v_min = max(v - range_value, 0)
228
229         # Détermination des bornes supérieures, en respectant les valeurs maximales HSV (H: 179, S/V: 255)
230         h_max = min(h + range_value, 179) # La teinte (H) varie de 0 à 179 en OpenCV
231         s_max = min(s + range_value, 255) # La saturation (S) varie de 0 à 255
232         v_max = min(v + range_value, 255) # La valeur (V) varie de 0 à 255
233
234         # Définition des plages de couleurs HSV pour le filtrage
235         low_color = np.array([h_min, s_min, v_min], dtype=int)
236         high_color = np.array([h_max, s_max, v_max], dtype=int)
237
238         # Conversion de l'image originale en format HSV
239         hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
240
241         # Création du masque en fonction des bornes HSV définies
242         mask = cv2.inRange(hsv_frame, low_color, high_color)
243
244         # Application du masque à l'image originale pour afficher uniquement les pixels filtrés
245         result_frame = cv2.bitwise_and(frame, frame, mask=mask)
246
247         # Fusionner l'image originale et l'image filtrée verticalement pour affichage
248         combined = cv2.concatenate([frame, result_frame])
249
250         # Afficher l'image mise à jour dans la fenêtre OpenCV
251         cv2.imshow(camera_name, combined)
252
253         # Quitter la boucle si l'utilisateur appuie sur la touche 'Échap' (code ASCII 27)
254         if cv2.waitKey(1) == 27:
255             break
256
257         # Fermer toutes les fenêtres OpenCV avant de quitter
258         cv2.destroyAllWindows()
259
260         # Retourner les bornes HSV utilisées pour le filtrage
261         return low_color, high_color
```

Transfert_donnees_lanceur.py

```
Transfert_donnees_lanceur.py > ...
1  """
2  Nom du fichier : Transfert_donnees_lanceur.py
3  Auteur : CAPRON Aurélien
4  Date : 15/02/2025
5  Description :
6      Ce script permet d'établir une connexion série via un port USB pour communiquer avec un dispositif externe, comme un lanceur.
7      Il vérifie la disponibilité du port USB et tente une connexion avec des paramètres de communication spécifiés.
8  """
9
10 import serial
11 import serial.tools.list_ports
12
13 def connection_port(port_usb):
14     """
15     Vérifie si le port USB spécifié est disponible parmi les ports série présents sur la machine.
16     """
17     # Vérification des ports disponibles
18     ports_disponibles = [port.device for port in serial.tools.list_ports.comports()]
19     if port_usb not in ports_disponibles:
20         print(f"\nPort \033[34m{port_usb}\033[0m non trouvé\n")
21         print("Ports disponibles :")
22         for port in ports_disponibles: # Affichage des ports disponibles
23             print(f"= {port}")
24         print()
25         return False # Le port spécifié n'est pas disponible
26     return True # Le port spécifié est disponible
27
28 def connexion_successful(port_usb, baud_rate):
29     """
30     Tente de se connecter au port USB spécifié avec le débit en bauds donné. Si la connexion échoue, un message d'erreur est affiché.
31     """
32     try:
33         # Tente d'ouvrir le port série avec les paramètres spécifiés
34         ser = serial.Serial(port_usb, baud_rate, timeout=1)
35         print(f"\nConnexion établie sur \033[34m{port_usb}\033[0m\n")
36     except serial.SerialException as error:
37         # Si la connexion échoue, un message d'erreur est affiché
38         print(f"\nImpossible de se connecter au port USB : \n\033[31m{error}\033[0m\n")
39         exit(1) # Arrêt du programme en cas d'échec de la connexion
40     return ser # Retourne l'objet Serial pour la communication série
```

II. Code Arduino

```
60 void loop() {
61     static String inputString = ""; // Stocke la chaîne de caractères reçue
62     static bool newData = false;    // Indique si une nouvelle donnée est
        disponible
63
64     // Lecture du message série caractère par caractère
65     while (Serial.available() > 0) {
66         char receivedChar = Serial.read();
67
68         if (receivedChar == '\n') { // Fin de message détectée
69             newData = true;
70             break;
71         } else {
72             inputString += receivedChar; // Ajout du caractère reçu à la chaîne
73         }
74     }
75 }
```

```
76 if (newData) {
77     // Conversion de la chaîne String en tableau de caractères
78     inputString.toCharArray(message, MAX_LEN_MESSAGE);
79
80     // Extraction des valeurs à partir du message reçu
81     if (sscanf(message, "%d:%d:%d:%d:%d", &azimuth, &altitude, &power, &
        frequencyThrow, &frequencyLauncher, &level) == 6) {
82
83         if (currentMillis - previousMillis >= Lfrequency) {
84             previousMillis = currentMillis; // Mise à jour du dernier temps
                d'exécution
85
86             // Mise à jour de la position du servomoteur en fonction des angles
                reçus
87             SERV01.write(45 - azimuth);
88             SERV02.write(45 - altitude);
89         }
90
91         // Filtrage des interférences sur la fréquence
92         if ((frequencyThrow == 0) || (frequencyThrow > 99)) {
93             Tfrequency = frequencyThrow;
94         }
95
96         currentMillis = millis(); // Obtention du temps actuel
97         // Filtrage des interférences sur la fréquence
98         if ((frequencyLauncher == 0) || (frequencyLauncher > 99)) {
99             Lfrequency = frequencyLauncher;
100         }
101
102         // Mise à jour du niveau si aucune valeur précédente
103         if (Level == 0) {
104             Level = level;
105         }
106     } else {
107         Serial.println("Erreur de format"); // Message d'erreur si le format est
            incorrect
108     }
109
110     // Clignotement de la LED pour indiquer la réception d'un message
111     digitalWrite(ledPinInfo, HIGH);
112     delay(50);
113     digitalWrite(ledPinInfo, LOW);
114
115     // Réinitialisation des variables pour la prochaine lecture
116     inputString = "";
117     newData = false;
118 }
```